

Workshop 8: Node.js & Express REST API Basics

Building Backend APIs with Node.js and Express

เป้าหมาย

Workshop นี้ให้นักศึกษาฝึกสร้าง Backend ด้วย Node.js และ Express.js โดยเน้น

- การสร้าง Node.js Project
- การติดตั้ง Express.js
- การสร้าง Web Server
- Routing
- Middleware
- RESTful API
- HTTP Methods: GET, POST, PUT และ DELETE
- การทดสอบ API ด้วย Postman

นักศึกษาจะสร้างโปรเจกต์ชื่อ

myapp

และสร้างไฟล์หลัก

app.js

งานที่ 1 — Create Your First Express Server

สร้าง Node.js Project และ Web Server ด้วย Express.js

Step 1 — สร้าง Project

เปิด Terminal แล้วพิมพ์

```
mkdir myapp  
cd myapp
```

จากนั้นสร้าง Node.js Project

```
npm init -y
```

ติดตั้ง Express

```
npm install express
```

Step 2 — สร้างไฟล์ app.js

สร้างไฟล์ชื่อ

app.js

แล้วเขียน Code

```
const express = require("express");

const app = express();

const port = 3000;

app.get("/", (req, res) => {

    res.send("Hello, World!");

});

app.listen(port, () => {

    console.log(
        "Server is running at http://localhost:3000"
    );

});
```

Step 3 — Run Server

เปิด Terminal แล้วพิมพ์

```
node app.js
```

จากนั้นเปิด Browser ที่

http://localhost:3000

ควรเห็นข้อความ

Hello, World!

ทดลองเพิ่ม Route

เพิ่ม Route ใหม่

```
app.get("/about", (req, res) => {  
  
    res.send("About Page");  
  
});
```

จากนั้นเปิด

<http://localhost:3000/about>

สิ่งที่ต้องใช้

- Node.js
- npm
- Express.js
- require()
- app.get()
- res.send()
- app.listen()

งานที่ 2 — Create GET and POST API

ให้นักศึกษาสร้าง REST API สำหรับจัดการข้อมูลสินค้าแบบง่าย
โดยจำลอง Database ด้วย Array

Step 1 — Middleware

เพิ่มคำสั่ง

```
app.use(express.json());
```

Middleware นี้ทำให้ Express สามารถอ่านข้อความ JSON ที่ส่งมาจาก Client ได้

Step 2 — สร้างข้อมูลเริ่มต้น

เพิ่ม Array

```
let items = [  
  { id: 1, name: "Item 1" },  
  { id: 2, name: "Item 2" }  
];
```

Step 3 — GET Request

สร้าง Route

```
app.get("/items", (req, res) => {  
  
  res.json(items);  
  
});
```

เปิด Browser หรือ Postman ที่

<http://localhost:3000/items>

ผลลัพธ์ควรเป็น

```
[  
  {  
    "id": 1,  
    "name": "Item 1"  
  },  
  {  
    "id": 2,
```

```
    "name": "Item 2"
  }
]
```

Step 4 — POST Request

สร้าง Route

```
app.post("/items", (req, res) => {

  const newItem = {
    id: items.length + 1,
    name: req.body.name
  };

  items.push(newItem);

  res.status(201).json(newItem);

});
```

ทดสอบด้วย Postman

กำหนด Method เป็น

POST

URL

`http://localhost:3000/items`

เลือก

Body → raw → JSON

แล้วส่งข้อมูล

```
{
  "name": "New Item"
}
```

ควรได้รับ Response เช่น

```
{
  "id": 3,
  "name": "New Item"
}
```

แนวคิด RESTful API

ใน Workshop นี้

- GET = อ่านข้อมูล
- POST = เพิ่มข้อมูลใหม่
- PUT = แก้ไขข้อมูล
- DELETE = ลบข้อมูล

งานที่ 3 — Update and Delete Items

ให้นักศึกษาเพิ่ม API สำหรับแก้ไขและลบข้อมูล

Step 1 — PUT Request

สร้าง Route

```
app.put("/items/:id", (req, res) => {

  const id = Number(req.params.id);

  const item =
    items.find(item => item.id === id);

  if (!item) {

    return res.status(404).json({
      message: "Item not found"
    });

  }

  item.name = req.body.name;

  res.json(item);

});
```

ทดสอบ PUT ด้วย Postman

กำหนด Method เป็น

PUT

ตัวอย่าง URL

`http://localhost:3000/items/1`

ส่ง JSON

```
{
  "name": "Updated Item"
}
```

จากนั้นใช้ GET

`http://localhost:3000/items`

เพื่อตรวจสอบว่าข้อมูลถูกแก้ไขแล้ว

Step 2 — DELETE Request

เพิ่ม Route

```
app.delete("/items/:id", (req, res) => {

  const id = Number(req.params.id);

  const index =
    items.findIndex(item => item.id === id);

  if (index === -1) {

    return res.status(404).json({
      message: "Item not found"
    });

  }

  const deletedItem =
    items.splice(index, 1);

  res.json(deletedItem[0]);

});
```

ทดสอบ DELETE

ใน Postman เลือก Method

DELETE

ตัวอย่าง URL

`http://localhost:3000/items/2`

จากนั้นใช้ GET อีกครั้ง

GET `http://localhost:3000/items`

เพื่อตรวจสอบว่าข้อมูลถูกลบแล้ว

ความหมายของ :id

ใน Route

```
"/items/:id"
```

:id คือ Route Parameter
สามารถอ่านค่าได้ด้วย

```
req.params.id
```

ตัวอย่าง

/items/5

จะทำให้

```
req.params.id
```

มีค่าเป็น

5

Code รวม

ไฟล์ app.js สามารถรวมเป็น

```
const express = require("express");

const app = express();
const port = 3000;

app.use(express.json());

let items = [
  { id: 1, name: "Item 1" },
  { id: 2, name: "Item 2" }
];

app.get("/", (req, res) => {
  res.send("Hello, World!");
});

app.get("/items", (req, res) => {
  res.json(items);
});

app.post("/items", (req, res) => {

  const newItem = {
    id: items.length + 1,
    name: req.body.name
  };

  items.push(newItem);

  res.status(201).json(newItem);
});

app.put("/items/:id", (req, res) => {

  const id = Number(req.params.id);

  const item =
    items.find(item => item.id === id);
```

```

    if (!item) {

        return res.status(404).json({
            message: "Item not found"
        });

    }

    item.name = req.body.name;

    res.json(item);
});

app.delete("/items/:id", (req, res) => {

    const id = Number(req.params.id);

    const index =
        items.findIndex(item => item.id === id);

    if (index === -1) {

        return res.status(404).json({
            message: "Item not found"
        });

    }

    const deletedItem =
        items.splice(index, 1);

    res.json(deletedItem[0]);
});

app.listen(port, () => {

    console.log(
        "Server is running at http://localhost:3000"
    );
});

```

});

Checklist ก่อนส่งงาน

1. Express Server

- ☐ สร้าง Project myapp
- ☐ ใช้ `npm init -y`
- ☐ ติดตั้ง Express
- ☐ มีไฟล์ `app.js`
- ☐ Server ทำงานที่ Port 3000
- ☐ เปิด `localhost:3000` ได้

2. GET และ POST

- ☐ ใช้ `express.json()`
- ☐ มี Array items
- ☐ GET `/items` ทำงานได้
- ☐ POST `/items` ทำงานได้
- ☐ รับข้อมูลจาก `req.body`
- ☐ ส่ง Response แบบ JSON ได้

3. PUT และ DELETE

- ☐ ใช้ Route Parameter `:id`
- ☐ ใช้ `req.params.id`
- ☐ PUT สามารถแก้ไขข้อมูลได้
- ☐ DELETE สามารถลบข้อมูลได้
- ☐ ทดสอบ API ด้วย Postman

งานที่นักศึกษาต้องส่ง

ส่ง Folder

myapp

โดยอย่างน้อยต้องประกอบด้วย

- app.js
- package.json
- package-lock.json

ไม่จำเป็นต้องส่ง Folder

node_modules

Node.js → Express → Routing → REST API → GET / POST / PUT / DELETE